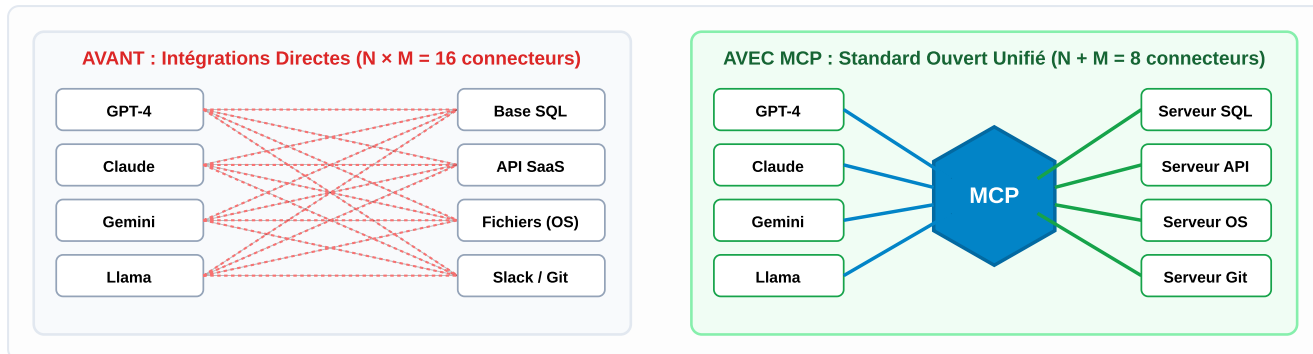


PROTOCOLE DE CONTEXTE DE MODÈLE

Architecture & Plomberie — De Zéro à la Production

1 LE PROBLÈME RÉSOLU : ÉLIMINATION DU SPAGHETTI \$N \backslash TIMES M\$



L'essence de MCP : Le protocole agit comme le *port USB-C des agents IA*. Les développeurs n'ont plus à concevoir des adaptateurs pour chaque SDK de modèle : ils exposent leurs données une seule fois dans un serveur MCP standardisé.

2 ANATOMIE : HÔTE, CLIENT ET SERVEUR

■ L'HÔTE (Application IA)

L'environnement applicatif complet dans lequel l'utilisateur interagit : **Claude Desktop**, **Cursor**, ou **Claude Code**. Il orchestre l'expérience utilisateur et gère la sécurité globale.

■ LE CLIENT (Dans l'Hôte)

Composant logiciel géré par l'Hôte. *Chaque serveur connecté possède son propre client dédié.* Il traduit les requêtes en messages normalisés JSON-RPC 2.0.

■ LE SERVEUR (Capacités)

Processus léger exposant des fonctionnalités via des primitives strictes. Il peut s'exécuter localement sur l'ordinateur ou à distance sur une infrastructure Cloud.

3 LES TROIS PRIMITIVES FONDAMENTALES DU PROTOCOLE

1. OUTILS (Tools)

CONTRÔLÉ PAR LE MODÈLE

Fonctions exécutables ayant des effets de bord. Le LLM analyse le prompt et décide d'invoquer ces fonctions de façon autonome.

Exemples : `recherche_bdd()`, `envoi_mail()`, `deploy_prod()`

2. RESSOURCES

CONTRÔLÉ PAR L'APPLICATION

Données passives en lecture seule injectées contextuellement par l'application pour enrichir la fenêtre de contexte.

Exemples : Schémas de base de données, logs système, contenu de fichiers sources.

3. INVITES (Prompts)

CONTRÔLÉ PAR L'UTILISATEUR

Modèles d'instructions pré-packagés et paramétrables invoqués explicitement par l'utilisateur via une commande interface.

Exemples : `/resumer`, `/revision-pr`, `/audit-securite`

4 COUCHE RÉSEAU, JSON-RPC 2.0 & CYCLE DE VIE

Transports Pris en Charge

• STDIO (Standard Input/Output) :

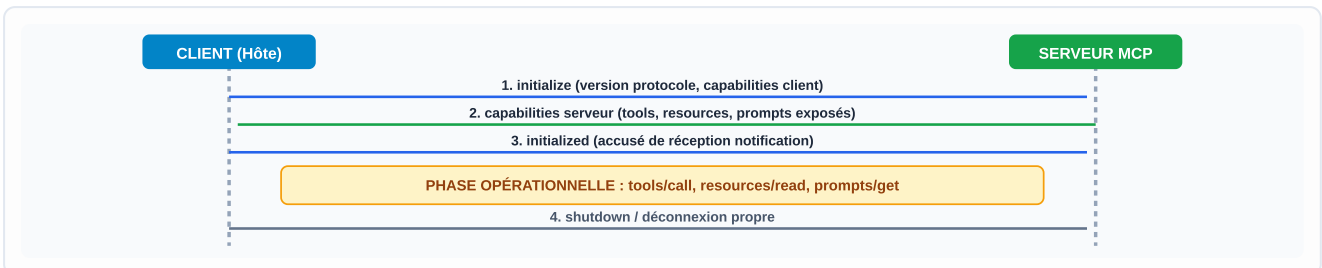
L'hôte exécute le serveur comme sous-processus local. Ultra-sécurisé, rapide, sans exposition de port réseau (idéal pour Git, FS).

• Streamable HTTP :

Standard pour serveurs cloud distants (remplaçant SSE). Idéal pour services SaaS centralisés (GitHub, Notion, Datadog).

Sur le Câble : Message JSON-RPC 2.0

```
{ "jsonrpc": "2.0", "id": 42, "method": "tools/call", "params":  
  { "name": "query_database", "arguments": { "sql":  
    "SELECT * FROM users LIMIT 10;" } } }
```



5 CE QUE PERSONNE NE VOUS DIT : LA RÉALITÉ EN PRODUCTION

⚠ Taxe sur les Jetons (Token Tax)

Chaque schéma d'outil est injecté dans le contexte du modèle.
50 outils × 50 tokens = 2 500 tokens de schéma consommés à chaque échange avant même le prompt de l'utilisateur !

🔑 Dispersion OAuth & Secrets

Déployer N serveurs distants impose une rotation éclatée des jetons d'accès et des secrets d'API, augmentant la charge d'administration et la surface d'attaque.

🔄 Dérive de Schéma (Schema Drift)

Si les paramètres d'un serveur distant changent sans avertissement, les agents exécutant d'anciens modèles généreront silencieusement des erreurs d'appel.

🔍 Angle Mort d'Observabilité

Le traçage distribué bout en bout (APM / OpenTelemetry) à travers des sous-processus locaux stdio reste un défi technique à consolider.

6 MATRICE COMPARATIVE : MCP VS SOLUTIONS ALTERNATIVES

Critère Décisionnel	Standard MCP	Function Calling Natif	Frameworks (LangChain)
Verrouillage Fournisseur	Faible (Ouvert)	Élevé (Spécifique)	Moyen (SDK-bound)
Portabilité des Outils	Maximale (Inter-hôtes)	Nulle (1 seul LLM)	Intermédiaire
Complexité d'Exécution	Moyenne (Processus)	Minime (Requête HTTP)	Moyenne à Forte
Pérennité & Écosystème	Standard de Fait	Dépendant Provider	Fragmentation

Règle d'or de l'ingénieur : MCP n'est pas une formule magique, c'est de la plomberie d'infrastructure propre. Utilisez-le dès lors que vos outils doivent être *partagés* entre plusieurs interfaces ou modèles sans réécrire d'adaptateurs.